

TOP 3 MICROSERVICES INTERVIEW QUESTIONS — Complete Analysis

For 16-Year Experience Professional

QUESTION 1: HOW TO CONVERT MONOLITH TO MICROSERVICES

Quick Analysis (30-Second Answer)

"Convert incrementally using the Strangler Fig Pattern. Never big bang. Start with identifying bounded contexts, extract one service at a time, use anti-corruption layer, and gradually redirect traffic. The goal is zero downtime and continuous delivery throughout migration."

Detailed Answer (5-Minute Interview Response)

Phase 1: Assessment & Planning (Weeks 1-4)

MIGRATION ASSESSMENT FRAMEWORK

Step 1: Identify Bounded Contexts (DDD)

Look for:

- Naturally isolated business capabilities
- Teams already organized around functions
- Database tables with clear ownership
- Low cross-domain transaction frequency

Step 2: Score Candidates for Extraction

Criteria:

- Business value (high = extract early)
- Technical complexity (low = extract early)
- Independence from monolith (high = extract early)
- Team ownership (clear = extract early)

Step 3: Understand Dependencies

Tools:

- Code analysis (JDepend, SonarQube)
- Runtime tracing (distributed tracing)
- Database foreign key analysis
- Team interviews

Phase 2: Strangler Fig Pattern Implementation

STRANGLER FIG – STEP BY STEP

STEP 1: Add Proxy Layer

Client → Proxy/API Gateway → Monolith
(routes all traffic)

STEP 2: Extract First Service + Redirect

Client → Proxy └─→ Payment Service (new)
 └─→ Monolith (others)

- /payments/* → Payment Service
- /* → Monolith

STEP 3: Extract More Services

Client → Proxy └─→ Payment Service
 └─→ Order Service
 └─→ Inventory Service
 └─→ Monolith (shrinking)

STEP 4: Decommission Monolith

Client → Proxy └─→ Payment Service
 └─→ Order Service
 └─→ Inventory Service
 └─→ Shipping Service

Monolith: DECOMMISSIONED ✓

Concrete Example: E-Commerce Migration

Before — Monolith Structure

```
// MONOLITH: Single application with all features
@SpringBootApplication
public class ECommerceMonolith {
    // Contains: Order, Payment, Inventory, Shipping, User, Review
}

// Single database with all tables
// orders, payments, inventory, shipments, users, reviews

// Tightly coupled code
@Service
public class OrderService {
    @Autowired
    private PaymentService paymentService; // Direct dependency

    @Autowired
    private InventoryService inventoryService; // Direct dependency

    public Order createOrder(OrderRequest request) {
        // All in one transaction
        inventoryService.reserveStock(request.getItems());
        paymentService.processPayment(request.getPayment());
        Order order = orderRepository.save(createOrder(request));
        shippingService.createShipment(order);
        return order;
    }
}
```

Step 1: Extract Payment Service

```

// NEW PAYMENT SERVICE (Independent Microservice)
@SpringBootApplication
@EnableDiscoveryClient
public class PaymentServiceApplication {
    // Own database: payment_db
}

@RestController
@RequestMapping("/api/payments")
public class PaymentController {

    @PostMapping("/process")
    public PaymentResponse process(@RequestBody PaymentRequest request) {
        // Payment logic only
        return paymentService.process(request);
    }

    @PostMapping("/refund")
    public RefundResponse refund(@RequestBody RefundRequest request) {
        return paymentService.refund(request);
    }
}

// MODIFIED MONOLITH – Call Payment Service via HTTP
@Service
public class OrderService {

    // Remove direct dependency, add HTTP client
    private final PaymentClient paymentClient; // Feign/RestClient

    public Order createOrder(OrderRequest request) {
        // Call Payment Service via API
        PaymentResponse payment = paymentClient.process(request.getPayment());

        // Still in monolith
        inventoryService.reserveStock(request.getItems());
        Order order = orderRepository.save(createOrder(request));
        shippingService.createShipment(order);
        return order;
    }
}

```

Step 2: Anti-Corruption Layer (ACL)

```
// ANTI-CORRUPTION LAYER – Protects monolith from service changes
@Component
public class PaymentAntiCorruptionLayer {

    private final PaymentClient paymentClient;
    private final PaymentTransformer transformer;

    public PaymentResponse processPayment(MonolithPaymentRequest request) {
        // Transform from monolith model to service model
        PaymentServiceRequest serviceRequest =
transformer.toServiceModel(request);

        // Call service
        PaymentServiceResponse serviceResponse =
paymentClient.process(serviceRequest);

        // Transform back to monolith model
        return transformer.toMonolithModel(serviceResponse);
    }

    // Fallback when service is down
    public PaymentResponse fallbackPayment(MonolithPaymentRequest request) {
        return new PaymentResponse("PENDING", "Payment queued for processing");
    }
}
```

Step 3: Data Migration Strategy

```

// DATA MIGRATION – Dual Write Pattern
@Component
public class DataMigrationService {

    // Phase 1: Dual Write (Write to both)
    @Transactional
    public Order createOrder(OrderRequest request) {
        // Write to monolith database (legacy)
        Order order = monolithOrderRepository.save(createOrder(request));

        // Also write to new service via API
        CompletableFuture.runAsync(() -> {
            orderServiceClient.createOrder(order);
        });

        return order;
    }

    // Phase 2: Backfill Historical Data
    @Scheduled(cron = "0 0 2 * * *") // Daily at 2 AM
    public void backfillHistoricalData() {
        List<Order> oldOrders = monolithOrderRepository.findNotMigrated();
        for (Order order : oldOrders) {
            try {
                orderServiceClient.createOrder(order);
                order.setMigrated(true);
                monolithOrderRepository.save(order);
            } catch (Exception e) {
                log.error("Failed to migrate order {}", order.getId());
            }
        }
    }

    // Phase 3: Switch to New Service as Source of Truth
    @Transactional
    public Order createOrder(OrderRequest request) {
        // Write only to new service
        OrderResponse response = orderServiceClient.createOrder(request);

        // Optional: Write to monolith for rollback capability
        if (rollbackMode) {
            monolithOrderRepository.save(convert(response));
        }
    }
}

```

```
    }

    return convert(response);
  }
}
```

Common Pitfalls & Solutions

Pitfall	Symptom	Solution
Distributed transactions	Data inconsistency	Saga pattern + compensating transactions
Shared database	Tight coupling continues	Database per service + event-driven sync
Chatty APIs	Too many network calls	API composition + GraphQL/BFF
Fallback failures	Cascade failures	Circuit breaker + timeout + retry
Migration never ends	Monolith still running	Set hard deadline, incentivize completion

Interview Power Phrases

- "The Strangler Fig Pattern isn't just about technical migration—it's about business continuity. Each extracted service must provide value independently, so stakeholders see progress."
- "The Anti-Corruption Layer is your most important safety net. It isolates the monolith from changes in extracted services, allowing independent evolution."
- "Database migration is harder than code migration. Use the Dual Write pattern: write to both, verify consistency, then switch readers, finally stop writing to legacy."

QUESTION 2: PATTERNS IN MICROSOFT SERVICES

Quick Analysis (30-Second Answer)

"Microservices patterns fall into five categories: Decomposition (Database per Service, Bounded Context), Integration (API Gateway, Service Discovery, Circuit Breaker), Data Management (Saga, CQRS, Event Sourcing), Observability (Distributed Tracing, Log Aggregation), and Cross-cutting (Sidecar, Ambassador, Externalized Configuration)."

Detailed Answer — Complete Pattern Catalog

Pattern 1: Decomposition Patterns

DECOMPOSITION PATTERNS

1. DECOMPOSE BY BUSINESS CAPABILITY

Split based on business functions:

E-Commerce → Order, Payment, Inventory, Shipping

Banking → Account, Transaction, Loan, Customer

✓ Aligns with business structure

✓ Clear ownership

✗ May cause chatty APIs if domains are coupled

2. DECOMPOSE BY SUBDOMAIN (DDD)

- Identify Bounded Contexts

- Define Ubiquitous Language

- Map Context Maps

Insurance → Policy, Claims, Underwriting, Billing

✓ Most aligned with business logic

✓ Natural service boundaries

✗ Requires deep domain expertise

3. DECOMPOSE BY VERB/USE CASE

- Split by actions: Create, Update, Delete, Query

- CQRS naturally fits here

Example: OrderWrite Service, OrderRead Service

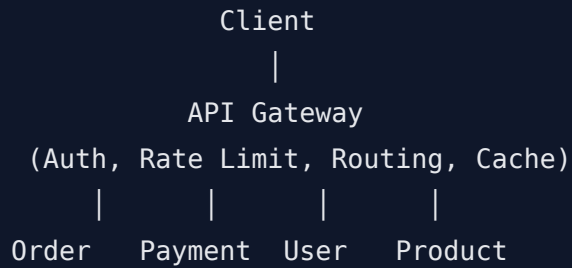
✓ Optimized scaling

✗ Complex consistency

Pattern 2: Integration Patterns

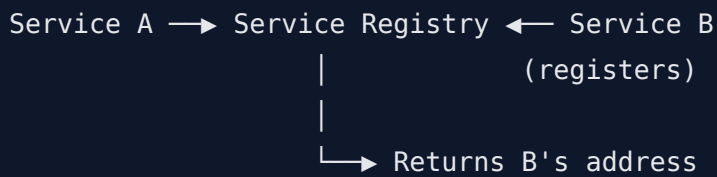
INTEGRATION PATTERNS

1. API GATEWAY



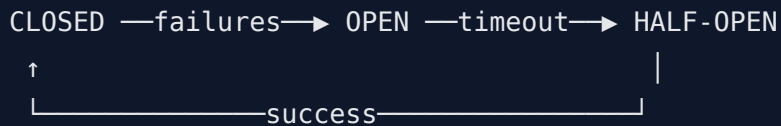
Use: Single entry point, cross-cutting concerns

2. SERVICE DISCOVERY



Use: Dynamic environments (K8s, cloud)

3. CIRCUIT BREAKER



Use: Prevent cascading failures

4. RETRY WITH BACKOFF

Attempt 1: — 100ms → Fail
Attempt 2: — 200ms → Fail

```
| | Attempt 3: — 400ms → Success ✓
```

```
| | Use: Transient failures (network, timeout)
```

Pattern 3: Data Management Patterns

DATA MANAGEMENT PATTERNS

1. DATABASE PER SERVICE

Order Service → order_db (PostgreSQL)
Payment Service → payment_db (PostgreSQL)
User Service → user_db (MongoDB)

- ✓ Independent scaling, technology choice
- ✗ Distributed transactions

2. SAGA PATTERN (Choreography)

Order Created —event→ Payment Service
|
Payment Success
|
▼
Inventory Service
|
Stock Reserved
|
▼
Shipping Service

- ✓ Decentralized, scalable
- ✗ Hard to debug, no central view

3. SAGA PATTERN (Orchestration)

Saga Orchestrator
|
├──
| ▼ ▼ ▼
Order Svc Payment Svc Inventory Svc

Compensating Actions
(refund, release, cancel)

- ✓ Central view, easier to manage
- ✗ Single point of coordination

4. CQRS (Command Query Responsibility Segregation)

Command → Write DB (Normalized)

|
| Event



Event Processor (Kafka)



Query → Read DB (Denormalized)

- ✓ Optimized reads/writes independently
- ✗ Eventual consistency, duplication

Pattern 4: Observability Patterns

OBSERVABILITY PATTERNS

1. DISTRIBUTED TRACING

Trace ID: abc-123

Gateway (5ms) → Order (50ms) → Payment (200ms)

|

Span 1

|

Span 2

|

Span 3

Tools: Jaeger, Zipkin, OpenTelemetry

2. HEALTH CHECK API

GET /health

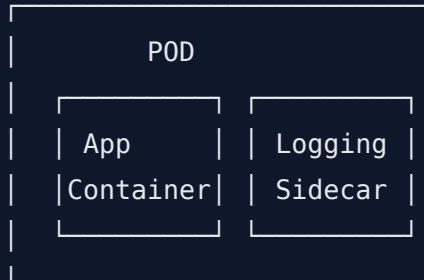
```
{
  "status": "UP",
  "components": {
    "database": { "status": "UP" },
    "redis": { "status": "UP" },
    "kafka": { "status": "DOWN" }
  }
}
```

Use: Kubernetes liveness/readiness probes

Pattern 5: Cross-Cutting Patterns

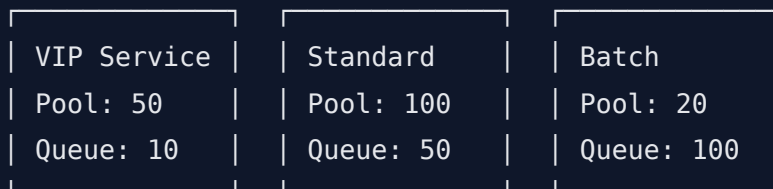
CROSS-CUTTING PATTERNS

1. SIDECAR PATTERN



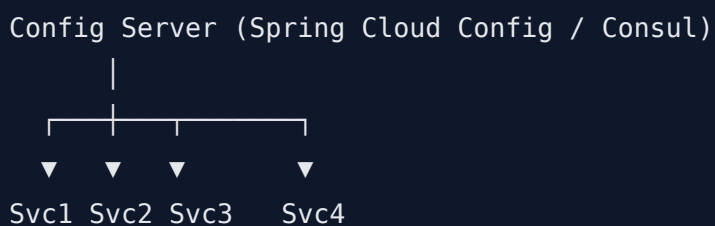
- ✓ Separation of concerns, reusable
- ✗ Resource overhead

2. BULKHEAD PATTERN



- ✓ Isolates failures, guarantees resources
- ✗ Complex configuration

3. EXTERNALIZED CONFIGURATION



- ✓ No hardcoded config, environment-specific

x Extra infrastructure

Complete Pattern Selection Guide

PATTERN SELECTION DECISION TREE

- Q1: How to split the monolith?
- Decompose by Business Capability
 - Decompose by Subdomain (if DDD expert available)
- Q2: How do services communicate?
- Need immediate response? Use Sync (API Gateway)
 - Can wait? Use Async (Event-Driven)
- Q3: How to handle service failures?
- Use Retry + Circuit Breaker + Timeout + Fallback
- Q4: How to manage distributed transactions?
- Simple workflow? Saga Choreography
 - Complex workflow? Saga Orchestration
- Q5: Read vs Write performance issues?
- Use CQRS + Event Sourcing
- Q6: Need to share cross-cutting concerns?
- Use Sidecar pattern
- Q7: How to find services dynamically?
- Service Discovery (Client-side or Server-side)

QUESTION 3: ADVANTAGES, DISADVANTAGES & ANTI-PATTERNS

Quick Analysis (30-Second Answer)

"Advantages: independent deployment, technology diversity, team autonomy, fault isolation, elastic scaling. Disadvantages: distributed system complexity, operational overhead, data consistency challenges, network latency, debugging difficulty. Anti-patterns: shared database, distributed monolith, too fine-grained, sync by default, ignoring fallbacks."

Detailed Answer — Complete Analysis

Advantages (Pros)

ADVANTAGES DETAILED

1. INDEPENDENT DEPLOYMENT

- ✓ Deploy Payment Service without touching Order
- ✓ Rollback single service
- ✓ Reduced deployment risk
- ✓ Faster time-to-market

Real Example: Amazon deploys every 11.7 seconds

2. TECHNOLOGY DIVERSITY

- ✓ Use Java for Order Service
- ✓ Use Go for Gateway (high concurrency)
- ✓ Use Python for ML Service
- ✓ Use Node.js for Real-time Service
- ✓ Choose right tool for each job

3. TEAM AUTONOMY & SCALING

- ✓ Each team owns service end-to-end
- ✓ No coordination between teams for features
- ✓ Scale teams independently (Amazon's two-pizza teams)
- ✓ Hire specialists per service

4. FAULT ISOLATION

Payment Service fails → Order Service degrades
Product Service → Still works!
User Service → Still works!

- ✓ Partial failures don't bring down whole system

5. ELASTIC SCALING

Black Friday: Scale Order Service to 200 instances

Normal day: Scale down to 10 instances

Payment Service: Always 50 instances

Reporting Service: 2 instances

✓ Scale only what needs scaling

✓ Cost optimization

6. SMALL CODEBASE

✓ Easier to understand (10K lines vs 500K)

✓ Faster builds (30 seconds vs 30 minutes)

✓ Faster onboarding

✓ Easier refactoring

Disadvantages (Cons)

DISADVANTAGES DETAILED

1. DISTRIBUTED SYSTEM COMPLEXITY

- x Network latency (10-100x slower than in-memory)
- x Partial failures
- x Network partitions (CAP theorem)
- x Distributed transactions

Example: Monolith: 5ms response → Microservices: 50ms

2. OPERATIONAL OVERHEAD

- x Need service discovery
- x Need API gateway
- x Need distributed tracing
- x Need log aggregation
- x Need circuit breakers
- x Need container orchestration

Team size needed:

Monolith: 5 developers

Microservices: 5 devs + 2 platform engineers

3. DATA CONSISTENCY CHALLENGES

- x No ACID transactions across services
- x Eventual consistency (stale reads)
- x Complex Saga patterns
- x Data duplication required (CQRS)

Example: Order created but inventory not reserved

Solution: Saga with 5 compensating actions

4. DEBUGGING COMPLEXITY

- x Stack traces across services

- x Hard to reproduce bugs
- x Multiple log files to search
- x Distributed tracing required

Monolith: `grep error.log`

Microservices: Search 50 services × 10 instances

5. TESTING COMPLEXITY

- x Integration tests across services
- x Contract testing needed
- x End-to-end tests are slow and flaky
- x Need test environments with all services

Time to run tests:

Monolith: 10 minutes

Microservices: 60+ minutes

6. VERSIONING & DEPENDENCY MANAGEMENT

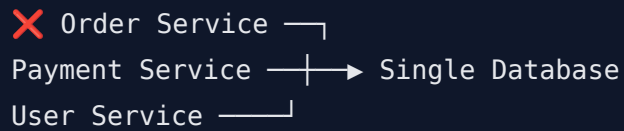
- x Service A uses API v1 of Service B
- x Service C uses API v2 of Service B
- x Need backward compatibility
- x Breaking changes require coordinated rollout

Solution: API versioning + consumer-driven contracts

Anti-Patterns (What NOT to Do)

ANTI-PATTERNS – AVOID THESE

ANTI-PATTERN 1: SHARED DATABASE



Problem: Tight coupling, single point of failure, no independent scaling, schema changes affect all services

Solution: Database per service + event-driven sync

ANTI-PATTERN 2: DISTRIBUTED MONOLITH

- ✗ Services split technically, not by business
- ✗ Deployed separately but must upgrade together
- ✗ Synchronous calls chain (A→B→C→D→E)

Symptoms:

- Changing one service requires changing 5 others
- Can't deploy independently
- High network latency

Solution: Proper bounded contexts, async where possible

ANTI-PATTERN 3: NANO-SERVICES (Too Fine-Grained)

- ✗ 200 services for a simple e-commerce site
- ✗ Each service has 2-3 endpoints
- ✗ Single request calls 15 services

Problem:

- High network overhead

- Hard to manage deployment
- Complex monitoring
- Team coordination overhead

Solution: Start with macro-services (10-50 lines)
Split only when necessary

ANTI-PATTERN 4: SYNCHRONOUS BY DEFAULT

❌ Order → Payment → Inventory → Shipping → Notification
(ALL SYNCHRONOUS)

Problem:

- User waits for entire chain
- Single failure kills whole request
- Low scalability

Solution: Async for non-critical path
(Notifications, emails, reporting)

ANTI-PATTERN 5: NO CIRCUIT BREAKER / RETRY

❌ Direct calls with no protection

Problem:

- Cascading failures
- Thread pool exhaustion
- Whole system goes down

Solution: Timeout + Retry + Circuit Breaker + Fallback

ANTI-PATTERN 6: VENDOR LOCK-IN

- ❌ All services use proprietary message bus
- ❌ All services use vendor-specific APIs

Problem: Can't migrate, expensive, vendor dictates

Solution: Use standards (OpenTelemetry, Prometheus,

Kafka vs proprietary)

ANTI-PATTERN 7: IGNORING FALLBACKS

❌ Payment fails → Order fails → User sees error

Better Approach:

- Payment fails → Mark order as "Payment Pending"
- Notify user, retry later
- Show partial data from cache

Solution: Always provide degraded but functional UI

Quick Reference Card

Aspect	Pro	Con	Anti-Pattern
Deployment	Independent	Complex orchestration	Distributed monolith
Data	Technology choice	Consistency	Shared database
Scale	Elastic per service	Network overhead	Nano-services
Resilience	Fault isolation	Partial failures	No circuit breaker
Performance	Parallel processing	Network latency	Sync by default
Debugging	Small codebase	Distributed traces	No observability

Interview Power Phrases

- "Microservices solve organizational problems more than technical ones. Conway's Law says system architecture mirrors communication structure—if your teams can't communicate, microservices won't fix that."
- "The biggest anti-pattern is the distributed monolith. You get the worst of both worlds: network latency AND coordinated deployments."

"Start with a monolith. Seriously. Extract services only when you have proven need—different scaling requirements, different teams, or different tech stacks."

"Database per service is non-negotiable. If services share a database, you don't have microservices—you have a distributed monolith with extra network calls."

"Fallbacks are not optional. Every service dependency must answer: 'What happens when this call fails?' If you can't answer, you're not production-ready."

Final Comparison: Monolith vs Microservices

MONOLITH VS MICROSERVICES – TRADE-OFFS		
DIMENSION	MONOLITH	MICROSERVICES
Deployment	Unified	Independent
Startup time	Slow (10 min)	Fast (30 sec)
Scalability	All or nothing	Per service
Tech stack	Single	Polyglot
Team structure	Centralized	Autonomous
Development speed	Fast initially	Fast at scale
Debugging	Easy (local)	Hard (distributed)
Consistency	Strong	Eventual
Network calls	0	Many
Operational cost	Low	High
Best for	Startups, MVP	Large orgs, scale